

Variance Inflation Factor in Python

Asked 6 years, 8 months ago Modified 1 year, 11 months ago Viewed 112k times Part of R Language Collective

I'm trying to calculate the variance inflation factor (VIF) for each column in a simple dataset in python:

62

```
a b c d
1 2 4 4
1 2 6 3
2 3 7 4
3 2 8 5
4 1 9 4
```

I have already done this in R using the vif function from the `usdm` library which gives the following results:

```
a <- c(1, 1, 2, 3, 4)
b <- c(2, 2, 3, 2, 1)
c <- c(4, 6, 7, 8, 9)
d <- c(4, 3, 4, 5, 4)

df <- data.frame(a, b, c, d)
vif_df <- vif(df)
print(vif_df)

Variables   VIF
a          22.95
b           3.00
c          12.95
d           3.00
```

However, when I do the same in python using the `statsmodel.vif` function, my results are:

```
a = [1, 1, 2, 3, 4]
b = [2, 2, 3, 2, 1]
c = [4, 6, 7, 8, 9]
d = [4, 3, 4, 5, 4]

ck = np.column_stack([a, b, c, d])

vif = [variance_inflation_factor(ck, i) for i in range(ck.shape[1])]
print(vif)

Variables   VIF
a          47.136986381369774
b          28.931586849315881
c          88.31586849315886
d          48.438356164383549
```

The results are vastly different, even though the inputs are the same. In general, results from the statsmodel VIF function seem to be wrong, but I'm not sure if this is because of the way I am calling it or if it is an issue with the function itself.

I was hoping someone could help me figure out whether I was incorrectly calling the statsmodel function or explain the discrepancies in the results. If it's an issue with the function then are there any VIF alternatives in python?

python r numpy statistics statsmodels Edit tags

Share Edit Follow Close Flag

asked Mar 7, 2017 at 21:09

Nizag
909 1 9 15

9 Answers

Sorted by
Reset to default Date modified (newest first)

Yet another solution. The following code gives the exact same VIF results as the R car package does.

```
def calc_reg_return_vif(x, y):
    """
    Utility function to calculate the VIF. This section calculates the linear
    regression inverse R squared.

    Parameters
    -----
    X : DataFrame
        Input data.
    y : Series
        Target.

    Returns
    -----
    vif : float
        Calculated VIF value.

    """
    X = X.values
    y = y.values

    if X.shape[1] == 1:
        print("Note, there is only one predictor here")
        X = X.reshape(-1, 1)
    reg = LinearRegression().fit(X, y)
    vif = 1 / (1 - reg.score(X, y))

    return vif

def calc_vif_from_scratch(df):
    """
    Calculating VIF using function from scratch

    Parameters
    -----
    df : DataFrame
        without target variable.

    Returns
    -----
    vif : DataFrame
        giving the feature - VIF value pair.

    """

    vif = pd.DataFrame()

    vif_list = []
    for feature in list(df.columns):
        y = df[feature]
        X = df.drop(feature, axis="columns")
        vif_list.append(calc_reg_return_vif(X, y))
    vif["feature"] = df.columns
    vif["VIF"] = vif_list
    return vif
```

I've tested it on the titanic dataset. You can get the full example here: <https://github.com/tulicsgabriel/Variance-Inflation-Factor-VIF->

Share Edit Follow Flag

answered Dec 1, 2021 at 23:00

bözgür
85 1 5

As mentioned by others and in [this post](#) by Josef Perktold, the function's author, `variance_inflation_factor` expects the presence of a constant in the matrix of explanatory variables. One can use `add_constant` from statsmodels to add the required constant to the dataframe before passing its values to the function.

73

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools.tools import add_constant

df = pd.DataFrame(
    {'a': [1, 1, 2, 3, 4],
     'b': [2, 2, 3, 2, 1],
     'c': [4, 6, 7, 8, 9],
     'd': [4, 3, 4, 5, 4]}
)

X = add_constant(df)
>>> pd.Series([variance_inflation_factor(X.values, i)
               for i in range(X.shape[1])],
              index=X.columns)
const    136.875
a         22.950
b           3.000
c         12.950
d           3.000
dtype: float64
```

I believe you could also add the constant to the right most column of the dataframe using `assign`:

```
X = df.assign(const=1)
>>> pd.Series([variance_inflation_factor(X.values, i)
               for i in range(X.shape[1])],
              index=X.columns)
a         22.950
b           3.000
c         12.950
d           3.000
const    136.875
dtype: float64
```

The source code itself is rather concise:

```
def variance_inflation_factor(exog, exog_idx):
    """
    exog : ndarray, (nobs, k_vars)
        design matrix with all explanatory variables, as for example used in
        regression
    exog_idx : int
        index of the exogenous variable in the columns of exog

    """
    k_vars = exog.shape[1]
    x_i = exog[:, exog_idx]
    mask = np.arange(k_vars) != exog_idx
    x_noti = exog[:, mask]
    r_squared_i = OLS(x_i, x_noti).fit().rsquared
    vif = 1. / (1 - r_squared_i)
    return vif
```

It is also rather simple to modify the code to return all of the VIFs as a series:

```
from statsmodels.regression.linear_model import OLS
from statsmodels.tools.tools import add_constant

def variance_inflation_factors(exog_df):
    """
    Parameters
    -----
    exog_df : dataframe, (nobs, k_vars)
        design matrix with all explanatory variables, as for example used in
        regression.

    Returns
    -----
    vif : Series
        variance inflation factors
    """
    exog_df = add_constant(exog_df)
    vifs = pd.Series(
        [1 / (1 - OLS(exog_df[col], exog_df.columns != col).values).fit().rsquared)
        for col in exog_df],
        index=exog_df.columns,
        name="VIF"
    )
    return vifs

>>> variance_inflation_factors(df)
const    136.875
a         22.950
b           3.000
c         12.950
dtype: float64
Name: VIF, dtype: float64
```

Per the solution of @T_T, one can also simply do the following:

```
vifs = pd.Series(np.linalg.inv(df.corr().to_numpy()).diagonal(),
                 index=df.columns,
                 name="VIF")
```

Share Edit Follow Flag

edited Oct 18, 2020 at 17:11

answered Feb 16, 2018 at 2:54

Alexander
106k 32 203 198

I think it is safe to add `X = add_constant(df.dropna())` in case of missing values – [steven](#) Feb 11, 2019 at 14:51

Thanks for this solution, I was extremely perplexed as to why I was getting such high VIF for my model's independent variables, which is how I ended up on this post. Much as I hate to do this, I'm almost tempted to complete my analysis in R – [hordle_buzz](#) Sep 21, 2020 at 14:53

With some data it is not possible to create the inverse matrix which leads to `LinAlgError: Singular matrix`. In this case replace `inv()` with `pinv()` compute the (Moore-Penrose) pseudo-inverse of a matrix. `pd.Series(np.linalg.pinv(X.corr()).to_numpy()).diagonal(), index=X.columns, name='VIF')` Out[13]: a 22.95 b 3.00 c 12.95 d 3.00 – [Wuuzzaa](#) Oct 7, 2021 at 13:25

37

3

Share

Edit

Follow

Flag

edited Aug 27, 2020 at 20:47

HoldOffHunger

19.2k 10 105 135

answered Mar 20, 2017 at 18:56

Drverzal

386 3 3

subtracting the mean from all variables would be similar. – Josef Mar 20, 2017 at 20:42

3

typo: column for constant should be filled with ones (not zeros). – Josef Mar 20, 2017 at 20:43

Good call on my typo. Edited my original post with the fix. – Drverzal Mar 20, 2017 at 21:04

That makes sense. Adding a column of 1s did the trick. Thanks! – Nizag Mar 21, 2017 at 20:24

Although it is already late, I am adding some modifications from the given answer. To get the best set after removing multicollinearity if we use @Chef1075 solution then we will lose the variables which are correlated. We have to remove only one of them. To do this I came with the following solution using @Steve answer:

3

3

Share

Edit

Follow

Flag

answered Apr 26, 2020 at 13:36

kasraful

59 1 8

Then, in this case, column d, c and b are the ones which does not cause multicollinearity, right? – AlSub Sep 28, 2021 at 22:18

1

@AlvaroMartinez. Right – kasraful Oct 8, 2021 at 9:42

@MdAsrafulKabir Can I ask why you are doing the following red_df_vif= df_vif.drop(df_vif.index[8]) ? So you calculate the VIF, order them highest to lowest; if the highest is greater than 5 then remove it and recalculate the entire process again? – Ciaran O'Brien Nov 16, 2021 at 16:31

@MdAsrafulKabir Can I ask why you are doing the following red_df_vif= df_vif.drop(df_vif.index[8]) ? So you calculate the VIF, order them highest to lowest; if the highest is greater than 5 then remove it and recalculate the entire process again? – Ciaran O'Brien Nov 16, 2021 at 16:32

2

3

Share

Edit

Follow

Flag

edited Feb 27, 2020 at 14:24

Huanfa Chen

607 8 17

answered Jul 13, 2018 at 16:35

Chef1075

2,614 9 40 57

Removing all variables with VIF higher than the thresh is INCORRECT. A correct approach is to remove the variable with the highest VIF, and then recalculate the VIF for the remaining variables, and repeat this step until no remaining variables have a VIF larger than thresh. For example, assuming x3=x2+x1, if we simply remove all variables with a high VIF, x1/x2/x3 would be removed and none of them is kept, and we might lose an important variable. – Huanfa Chen Oct 25, 2020 at 12:12

Yes, agree with Huanfa. @chef and others - you will be stopping more variables than you need to if you simply dropped all columns above your VIF threshold from your initial run. This needs to be done iteratively as mentioned by Huanfa. – veg2020 Aug 23, 2021 at 0:57

1

3

Share

Edit

Follow

Flag

edited Jan 24, 2020 at 15:54

Max Alonzo

21 3

here code using dataframe python:

1

17

3

1. Use formula in statasmodels:

import pandas as pd

import statasmodels.formula.api as smf

def get_vif(exogs, data):

"""Return VIF (variance inflation factor) DataFrame

Args:

exogs (list): list of exogenous/independent variables

data (DataFrame): the df storing all variables

Returns:

VIF and Tolerance DataFrame for each exogenous variable

Notes:

Assume we have a list of exogenous variable [X1, X2, X3, X4].

To calculate the VIF and Tolerance for each variable, we regress each of them against other exogenous variables. For instance, the regression model for X3 is defined as:

X3 = X2 + X2 + X4

And then we extract the R-squared from the model to calculate:

VIF = 1 / (1 - R-squared)

Tolerance = 1 - R-squared

The cutoff to detect multicollinearity:

VIF > 10 or Tolerance < 0.1

...

initialize dictionaries

vif_dict, tolerance_dict = {}, {}

create formula for each exogenous variable

for exog in exogs:

not_exog = [i for i in exogs if i != exog]

formula = f"{exog} = {' + ' + '.join(not_exog)}"

extract r-squared from the fit

r_squared = smf.ols(formula, data=data).fit().rsquared

calculate VIF

vif = 1/(1 - r_squared)

vif_dict[exog] = vif

calculate tolerance

tolerance = 1 - r_squared

In case you don't wanna deal with variance_inflation_factor and add_constant. Please consider the following two functions.

17

3

Share

Edit

Follow

Flag

edited Jan 24, 2020 at 15:23

Max Alonzo

21 3

1. Use formula in statasmodels:

import pandas as pd

import statasmodels.formula.api as smf

def get_vif(exogs, data):

"""Return VIF (variance inflation factor) DataFrame

Args:

exogs (list): list of exogenous/independent variables

data (DataFrame): the df storing all variables

Returns:

VIF and Tolerance DataFrame for each exogenous variable

Notes:

Assume we have a list of exogenous variable [X1, X2, X3, X4].

To calculate the VIF and Tolerance for each variable, we regress each of them against other exogenous variables. For instance, the regression model for X3 is defined as:

X3 = X2 + X2 + X4

And then we extract the R-squared from the model to calculate:

VIF = 1 / (1 - R-squared)

Tolerance = 1 - R-squared

The cutoff to detect multicollinearity:

VIF > 10 or Tolerance < 0.1

...

initialize dictionaries

vif_dict, tolerance_dict = {}, {}

create formula for each exogenous variable

for exog in exogs:

not_exog = [i for i in exogs if i != exog]

formula = f"{exog} = {' + ' + '.join(not_exog)}"

extract r-squared from the fit

r_squared = smf.ols(formula, data=data).fit().rsquared

calculate VIF

vif = 1/(1 - r_squared)

vif_dict[exog] = vif

calculate tolerance

tolerance = 1 - r_squared

https://stackoverflow.com/questions/4268370/variance-inflation-factor-in-python

23

```
tolerance_dict[exog] = tolerance

# return VIF DataFrame
df_vif = pd.DataFrame({'VIF': vif_dict, 'Tolerance': tolerance_dict})

return df_vif
```

2. Use LinearRegression in sklearn:

```
# import warnings
# warnings.simplefilter(action='ignore', category=FutureWarning)
import pandas as pd
from sklearn.linear_model import LinearRegression

def sklearn_vif(exogs, data):

    # initialize dictionaries
    vif_dict, tolerance_dict = {}, {}

    # form input data for each exogenous variable
    for exog in exogs:
        not_exog = [i for i in exogs if i != exog]
        X, y = data[not_exog], data[exog]

        # extract r-squared from the fit
        r_squared = LinearRegression().fit(X, y).score(X, y)

        # calculate VIF
        vif = 1 / (1 - r_squared)
        vif_dict[exog] = vif

        # calculate tolerance
        tolerance = 1 - r_squared
        tolerance_dict[exog] = tolerance

    # return VIF DataFrame
    df_vif = pd.DataFrame({'VIF': vif_dict, 'Tolerance': tolerance_dict})

    return df_vif
```

Example:

```
import seaborn as sns

df = sns.load_dataset('car_crashes')
exogs = ['alcohol', 'speeding', 'no_previous', 'not_distracted']

[In] %timeit -n 100
get_vif(exogs=exogs, data=df)

[Out]
      VIF  Tolerance
alcohol  3.436072  0.291938
no_previous  3.113984  0.321132
not_distracted  2.668456  0.374749
speeding  1.884348  0.538608

69.6 ms ± 8.96 ms per loop (mean ± std. dev. of 7 runs, 100 loops each)

[In] %timeit -n 100
sklearn_vif(exogs=exogs, data=df)

[Out]
      VIF  Tolerance
alcohol  3.436072  0.291938
no_previous  3.113984  0.321132
not_distracted  2.668456  0.374749
speeding  1.884348  0.538608

15.7 ms ± 1.4 ms per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

Share Edit Follow Flag

edited Jul 6, 2019 at 4:41

answered Feb 24, 2019 at 23:06

 **Steven**

2.219 20 38

-  The cutoff to detect multicollinearity: VIF > 10 or Tolerance < 0.1, you must change 0.2 in Tolerance < 0.2 by Tolerance < 0.1 – Sandra Guerrero Jun 4, 2019 at 21:35
-  @SandraGuerrero indeed that's a typo. – Steven Jun 5, 2019 at 4:16
-  Appreciate the effort put in to explain this. Thank you very much! – Sudheer Rao Mar 18, 2020 at 15:46

For future comers to this thread (like me):

```
import numpy as np
import scipy as sp

a = [1, 1, 2, 3, 4]
b = [2, 2, 3, 2, 1]
c = [4, 6, 7, 8, 9]
d = [4, 3, 4, 5, 4]

ck = np.column_stack([a, b, c, d])
cc = sp.corrcoef(ck, rowvar=False)
VIF = np.linalg.inv(cc)
VIF.diagonal()
```

This code gives

```
array([[22.95,  3. , 12.95,  3. ]])
```

[EDIT]

In response to a comment, I tried to use `DataFrame` as much as possible (`numpy` is required to invert a matrix).

```
import pandas as pd
import numpy as np

a = [1, 1, 2, 3, 4]
b = [2, 2, 3, 2, 1]
c = [4, 6, 7, 8, 9]
d = [4, 3, 4, 5, 4]

df = pd.DataFrame({'a':a, 'b':b, 'c':c, 'd':d})
df_cor = df.corr()
pd.DataFrame(np.linalg.inv(df_cor().values), index=df_cor.index,
              columns=df_cor.columns)
```

The code gives

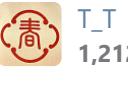
```
      a      b      c      d
a  22.950000  6.453681 -16.301917 -6.453681
b   6.453681  3.000000 -4.088441 -2.000000
c -16.301917 -4.088441 12.950000  4.088441
d   6.453681 -2.000000  4.088441  3.000000
```

The diagonal elements give VIF.

Share Edit Follow Flag

edited Feb 12, 2019 at 2:21

answered Jul 22, 2018 at 8:03

 **T.T**

1.212 16 22

- 1  Could you please add a solution for dataframe input instead of numpy array? – Steven Feb 11, 2019 at 15:26
- 2  Looks good. To just get the VIFs as a Series: `vifs = pd.Series(np.linalg.inv(df.corr().values).diagonal(), index=df_cor.index)` – Alexander Mar 15, 2019 at 16:59
-  vif is the diagonal elements of inverse corr matrix? edited Yes, check the link : documentation.statsoft.com/STATISTICAHelp.aspx?path=glossary/ – Shuai Liu Aug 14, 2019 at 1:58

Example for Boston Data:

VIF is calculated by auxiliary regression, so not dependent on the actual fit.

See below:

```
from patsy import Dmatrices
from statsmodels.stats.outliers_influence import variance_inflation_factor
import statsmodels.api as sm

# Break into left and right hand side; y and X
y, X = Dmatrices(formula="medv ~ crim + zn + nox + ptratio + black + rm", data=boston,
                  return_type="dataframe")

# For each Xi, calculate VIF
vif = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]

# Fit X to y
result = sm.OLS(y, X).fit()
```

Share Edit Follow Flag

answered Aug 18, 2017 at 6:22

 **S.M**

530 11 28